Assignment-1 - Report

Pushkar Gudla - AI24BTECH11012

Question 2: Universal Approximation Theorem

(a) Formal Statement

Let $K \subset \mathbb{R}^n$ be a compact set and let $f : K \to \mathbb{R}$ be a continuous function.

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous, non-polynomial activation function (e.g. sigmoid, tanh).

Then, for any $\varepsilon > 0$, there exists a single-hidden-layer feedforward neural network of the form

$$\hat{f}(x) = \sum_{i=1}^{N} a_i \, \sigma(w_i^\top x + b_i)$$

such that

$$\sup_{x \in K} |f(x) - \hat{f}(x)| < \varepsilon.$$

Specifically:

- **Target functions:** Continuous functions $f \in C(K)$
- **Domain:** Compact subset $K \subset \mathbb{R}^n$
- **Approximation notion:** Uniform approximation
- **Norm used:** Supremum norm $\| \cdot \|_\infty$

(b) Main Idea of the Proof

The key idea is that hidden-layer neurons act like *basis functions*.

- Sigmoid units can approximate localized step-like or bump functions.
- By linearly combining enough such units, the network can approximate arbitrary shapes.
- This connects to classical approximation results such as the Stone–Weierstrass theorem, which guarantees that certain function classes are dense in $C(K)$.

A full proof is constructive but not required for the theorem statement.

(c) Practical Limitations

1. No guarantee on network size

The theorem does not give bounds on the required number of hidden neurons. In practice, this may lead to extremely wide networks that are inefficient or impossible to train.

2. No guarantee of learnability

The theorem is purely existential. It does not ensure that gradient descent or backpropagation will find the required parameters, nor that training will be stable or efficient.

(d) Effect of ReLU and Increased Depth

With ReLU activations:

- Universal approximation still holds.
- Depth becomes more important than width.
- Deep but narrow ReLU networks can approximate functions more efficiently than shallow wide networks.

Thus, increasing depth improves expressive power while keeping the width relatively small.

Question 3: One Step of Gradient Descent in a Deep Neural Network

Network Setup

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad y = 1, \quad \eta = 0.1$$

$$W^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \; W^{(2)} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \; W^{(3)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$b^{(1)} = b^{(2)} = b^{(3)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Part A: Softmax with Cross-Entropy Loss

(a) Forward Pass

The input is

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Layer 1 (Affine + ReLU)

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Applying ReLU elementwise,

$$a^{(1)} = \mathrm{ReLU}(z^{(1)}) = \begin{bmatrix} \max(0,1) \\ \max(0,1) \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Layer 2 (Affine + ReLU)

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

Applying ReLU,

$$a^{(2)} = \begin{bmatrix} \max(0,0) \\ \max(0,2) \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

Layer 3 (Output Layer)

$$s = W^{(3)}a^{(2)} + b^{(3)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

The softmax probabilities are

$$q_i = \frac{e^{s_i}}{\sum_j e^{s_j}},$$

hence

$$q = \begin{bmatrix} \frac{e^0}{e^0+e^2} \\ \frac{e^2}{e^0+e^2} \end{bmatrix} = \begin{bmatrix} \frac{1}{1+e^2} \\ \frac{e^2}{1+e^2} \end{bmatrix}.$$

(b) Cross-Entropy Loss

The true class label is $y = 1$. For a single example, the cross-entropy loss is

$$L_{\mathrm{CE}} = -\log q_y.$$

Substituting $q_1 = \frac{1}{1+e^2}$,

$$L_{\mathrm{CE}} = -\log\left(\frac{1}{1+e^2}\right) = \log(1+e^2).$$

(c) Gradient w.r.t. Logits

For softmax followed by cross-entropy, the gradient w.r.t. logits is

$$\nabla_s L = q - e_y,$$

where e_y is the one-hot vector of the true class.

Here,

$$e_y = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Thus,

$$\nabla_s L = \begin{bmatrix} \frac{1}{1+e^2} - 1 \\ \frac{e^2}{1+e^2} \end{bmatrix} = \begin{bmatrix} -\frac{e^2}{1+e^2} \\ \frac{e^2}{1+e^2} \end{bmatrix}.$$

For convenience, define

$$\alpha = \frac{e^2}{1+e^2}.$$

(d) Gradients w.r.t. Weights and Biases

Layer 3 Using $s = W^{(3)} a^{(2)} + b^{(3)}$,

$$\nabla_{W^{(3)}} = \nabla_s L \, (a^{(2)})^\top = \begin{bmatrix} -\alpha \\ \alpha \end{bmatrix} \begin{bmatrix} 0 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -2\alpha \\ 0 & 2\alpha \end{bmatrix}.$$

$$\nabla_{b^{(3)}} = \nabla_s L = \begin{bmatrix} -\alpha \\ \alpha \end{bmatrix}.$$

Layer 2 Backpropagating to $a^{(2)}$,

$$\nabla_{a^{(2)}} = (W^{(3)})^\top \nabla_s L = \begin{bmatrix} -\alpha \\ \alpha \end{bmatrix}.$$

ReLU derivative:

$$\mathrm{ReLU}'(z^{(2)}) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \nabla_{z^{(2)}} = \nabla_{a^{(2)}} \odot \mathrm{ReLU}'(z^{(2)}) = \begin{bmatrix} 0 \\ \alpha \end{bmatrix}.$$

Thus,

$$\nabla_{W^{(2)}} = \nabla_{z^{(2)}} (a^{(1)})^\top = \begin{bmatrix} 0 & 0 \\ \alpha & \alpha \end{bmatrix}, \quad \nabla_{b^{(2)}} = \begin{bmatrix} 0 \\ \alpha \end{bmatrix}.$$

Layer 1

$$\nabla_{a^{(1)}} = (W^{(2)})^\top \nabla_{z^{(2)}} = \begin{bmatrix} \alpha \\ \alpha \end{bmatrix}.$$

Since $z^{(1)} > 0$ elementwise,

$$\nabla_{z^{(1)}} = \nabla_{a^{(1)}}.$$

Therefore,

$$\nabla_{W^{(1)}} = \nabla_{z^{(1)}} x^\top = \begin{bmatrix} \alpha & \alpha \\ \alpha & \alpha \end{bmatrix}, \quad \nabla_{b^{(1)}} = \begin{bmatrix} \alpha \\ \alpha \end{bmatrix}.$$

(e) Gradient Descent Update

All parameters are updated using

$$\theta_{\text{new}} = \theta - \eta \nabla_\theta, \quad \eta = 0.1.$$

Recall

$$\alpha = \frac{e^2}{1 + e^2}.$$

Layer 3

$$W^{(3)}_{\text{new}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} 0 & -2\alpha \\ 0 & 2\alpha \end{bmatrix} = \begin{bmatrix} 1 & 0.2\alpha \\ 0 & 1 - 0.2\alpha \end{bmatrix}.$$

$$b^{(3)}_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -\alpha \\ \alpha \end{bmatrix} = \begin{bmatrix} 0.1\alpha \\ -0.1\alpha \end{bmatrix}.$$

Layer 2

$$W^{(2)}_{\text{new}} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} 0 & 0 \\ \alpha & \alpha \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 1 - 0.1\alpha & 1 - 0.1\alpha \end{bmatrix}.$$

$$b^{(2)}_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} 0 \\ \alpha \end{bmatrix} = \begin{bmatrix} 0 \\ -0.1\alpha \end{bmatrix}.$$

Layer 1

$$W^{(1)}_{\text{new}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} \alpha & \alpha \\ \alpha & \alpha \end{bmatrix} = \begin{bmatrix} 1 - 0.1\alpha & -0.1\alpha \\ -0.1\alpha & 1 - 0.1\alpha \end{bmatrix}.$$

$$b^{(1)}_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} \alpha \\ \alpha \end{bmatrix} = \begin{bmatrix} -0.1\alpha \\ -0.1\alpha \end{bmatrix}.$$

Part B: Multiclass Hinge Loss

(a) Hinge Loss

The multiclass hinge loss is defined as

$$L_{\text{hinge}} = \sum_{j \neq y} \max(0, s_j - s_y + 1).$$

There are two classes and the true class is $y = 1$. Thus, only the term with $j = 2$ contributes:

$$L_{\text{hinge}} = \max(0, s_2 - s_1 + 1).$$

From the forward pass,

$$s = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \quad s_1 = 0,\ s_2 = 2.$$

Therefore,

$$L_{\text{hinge}} = \max(0, 2 - 0 + 1) = 3.$$

(b) Gradient w.r.t. Logits

Since the margin constraint is active ($s_2 - s_1 + 1 > 0$), the hinge loss is differentiable at this point.

For the active class $j = 2$,

$$\frac{\partial L}{\partial s_2} = 1, \quad \frac{\partial L}{\partial s_1} = -1.$$

Thus, the gradient w.r.t. the logits is

$$\nabla_s L = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

(c) Gradients w.r.t. Parameters

Layer 3 Using $s = W^{(3)} a^{(2)} + b^{(3)}$,

$$\nabla_{W^{(3)}} = \nabla_s L \, (a^{(2)})^\top = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ 0 & 2 \end{bmatrix}.$$

$$\nabla_{b^{(3)}} = \nabla_s L = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Layer 2 Backpropagating to $a^{(2)}$,

$$\nabla_{a^{(2)}} = (W^{(3)})^\top \nabla_s L = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

ReLU derivative:

$$\mathrm{ReLU}'(z^{(2)}) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \nabla_{z^{(2)}} = \nabla_{a^{(2)}} \odot \mathrm{ReLU}'(z^{(2)}) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Thus,

$$\nabla_{W^{(2)}} = \nabla_{z^{(2)}} (a^{(1)})^\top = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}, \quad \nabla_{b^{(2)}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Layer 1

$$\nabla_{a^{(1)}} = (W^{(2)})^\top \nabla_{z^{(2)}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Since $z^{(1)} > 0$ elementwise,

$$\nabla_{z^{(1)}} = \nabla_{a^{(1)}}.$$

Therefore,

$$\nabla_{W^{(1)}} = \nabla_{z^{(1)}} x^\top = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \nabla_{b^{(1)}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

(d) Gradient Descent Update

All parameters are updated using

$$\theta_{\text{new}} = \theta - \eta \nabla_\theta, \quad \eta = 0.1.$$

Layer 3

$$W^{(3)}_{\text{new}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} 0 & -2 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0.2 \\ 0 & 0.8 \end{bmatrix},$$

$$b^{(3)}_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}.$$

Layer 2

$$W^{(2)}_{\text{new}} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 0.9 & 0.9 \end{bmatrix},$$

$$b^{(2)}_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -0.1 \end{bmatrix}.$$

Layer 1

$$W_{\text{new}}^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.9 & -0.1 \\ -0.1 & 0.9 \end{bmatrix},$$

$$b_{\text{new}}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.1 \\ -0.1 \end{bmatrix}.$$

Question 5: ADALINE (Adaptive Linear Neuron)

5.1 Theory

(a) ADALINE Model and Learning Algorithm

The ADALINE (Adaptive Linear Neuron) model, introduced by Widrow and Hoff (1960), is a single-layer linear neuron defined by

$$y = w^\top x + b,$$

where $x \in \mathbb{R}^d$ is the input vector, $w \in \mathbb{R}^d$ is the weight vector, and $b \in \mathbb{R}$ is the bias.

Unlike the perceptron, ADALINE is trained by minimizing a *mean squared error* (MSE) objective rather than a classification error. For a dataset $\{(x_i, t_i)\}_{i=1}^N$, the loss function is

$$J(w, b) = \frac{1}{2} \sum_{i=1}^{N} (t_i - y_i)^2 = \frac{1}{2} \sum_{i=1}^{N} \big(t_i - (w^\top x_i + b)\big)^2,$$

where t_i denotes the target output.

Gradient Computation For a single training example (x_i, t_i), the gradients of the loss are

$$\frac{\partial J}{\partial w} = -(t_i - y_i)x_i, \quad \frac{\partial J}{\partial b} = -(t_i - y_i).$$

ADALINE Learning Rule (Widrow–Hoff Rule) Using gradient descent with learning rate $\eta > 0$, the parameter updates are

$$w \leftarrow w + \eta(t_i - y_i)x_i, \quad b \leftarrow b + \eta(t_i - y_i).$$

Learning Algorithm Given a training set $\{(x_i, t_i)\}_{i=1}^N$ and learning rate η:

1. Initialize weights $w^{(0)}$ and bias $b^{(0)}$
2. For each epoch $k = 0, 1, 2, \ldots$:
   (a) For each training sample (x_i, t_i):

i. Compute the linear output

$$y_i = (w^{(k)})^\top x_i + b^{(k)}.$$

ii. Compute the prediction error

$$e_i = t_i - y_i.$$

iii. Update the weights

$$w^{(k+1)} = w^{(k)} + \eta\, e_i\, x_i.$$

iv. Update the bias

$$b^{(k+1)} = b^{(k)} + \eta\, e_i.$$

3. Repeat until convergence of the loss.

The updates may be applied sequentially (stochastic gradient descent) or using all samples simultaneously (batch gradient descent).

(b) Convergence to the Least-Squares Solution

Let $X \in \mathbb{R}^{N\times d}$ denote the data matrix whose rows are the input vectors $x_i^\top$, and let $t \in \mathbb{R}^N$ be the target vector. The ADALINE objective function can be written compactly as

$$J(w) = \frac{1}{2}\|Xw - t\|_2^2.$$

Convexity of the Objective Expanding the objective,

$$J(w) = \frac{1}{2}w^\top X^\top X w - w^\top X^\top t + \frac{1}{2}t^\top t.$$

The gradient and Hessian are

$$\nabla J(w) = X^\top(Xw - t), \quad \nabla^2 J(w) = X^\top X.$$

Since $X^\top X$ is symmetric and positive semidefinite,

$$X^\top X \succeq 0,$$

the objective function $J(w)$ is convex in w. Consequently, any stationary point is a global minimizer.

Least-Squares Solution Setting the gradient to zero yields the normal equations:

$$\nabla J(w^\star) = 0 \quad \Longrightarrow \quad X^\top X w^\star = X^\top t.$$

If $X^\top X$ is invertible, the unique minimizer is

$$w^\star = (X^\top X)^{-1} X^\top t,$$

which is exactly the classical least-squares solution.

Gradient Descent Convergence The gradient $\nabla J(w)$ is Lipschitz continuous with Lipschitz constant

$$L = \lambda_{\max}(X^\top X),$$

where $\lambda_{\max}$ denotes the largest eigenvalue.

For any learning rate satisfying

$$0 < \eta < \frac{2}{L},$$

the gradient descent updates

$$w^{(k+1)} = w^{(k)} - \eta \nabla J(w^{(k)})$$

converge to the unique minimizer $w^\star$.

Independence from Linear Separability The convergence result depends only on the convexity of $J(w)$ and not on the separability of the data. Even when the classes are not linearly separable, ADALINE converges to the weight vector that minimizes the squared error.

This stands in contrast to the perceptron algorithm, whose convergence is guaranteed only when the data are linearly separable.

5.3 Experiments and Analysis

All experiments are conducted using the dataset described in Section 4. Features are standardized prior to training to ensure numerical stability of gradient descent.

(a) Dataset Visualization

The dataset is visualized by projecting the standardized feature vectors onto the first two principal components using PCA.

- PCA is used only for visualization.
- Color indicates the target variable (mess duration).

Figure 1 shows that the data is continuous and not linearly separable in the projected space, motivating the use of regression-based learning.

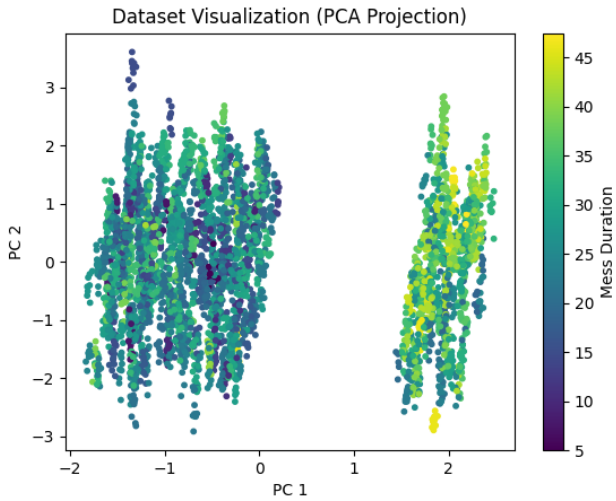

Figure 1: Dataset visualization using 2D PCA projection.

(b) Training Dynamics of ADALINE

ADALINE is trained using batch gradient descent. Figure 2 shows the mean squared error (MSE) as a function of training epochs.

- Training and validation errors decrease smoothly.
- No significant overfitting is observed.

This behavior is consistent with the convex quadratic objective optimized by ADALINE.

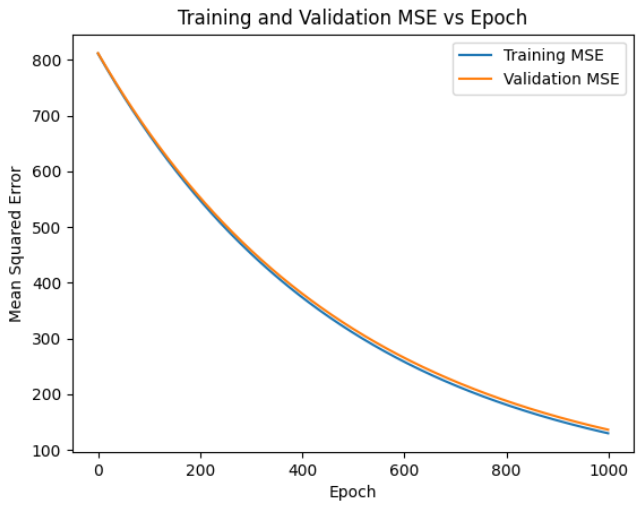

Figure 2: Training and validation MSE versus epoch.

Decision Function Visualization The learned decision function is visualized in the 2D PCA space. Although ADALINE is a linear model in the original feature space, the projection

induces a non-axis-aligned linear structure.

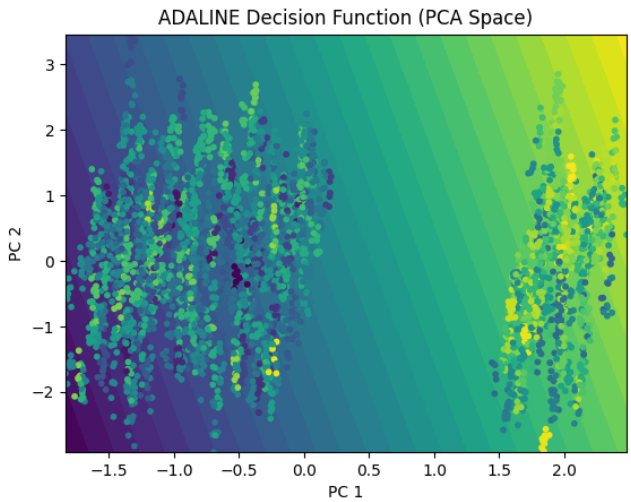

Figure 3: ADALINE decision function in PCA-projected space.

(c) Effect of Learning Rate

The effect of different learning rates $\eta \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ is shown in Figure 5.

- Small η: slow but stable convergence.
- Moderate η: fast convergence.
- Large η: unstable or divergent behavior.

This highlights the sensitivity of gradient descent to the choice of step size.

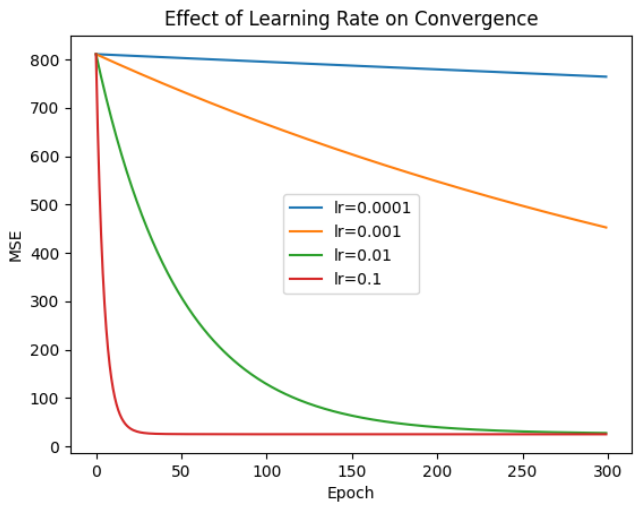

Figure 4: Convergence curves for different learning rates.

For very large learning rates (e.g., $\eta = 10$), the gradient descent updates become unstable and overshoot the optimal solution. As a result, the weights grow extremely large and the mean squared error explodes to very high values (on the order of 10^{300}), which dominates the scale of the plot. This causes the other curves corresponding to smaller learning rates to appear nearly flat near zero. The behavior illustrates that excessively large learning rates lead to divergence rather than convergence.

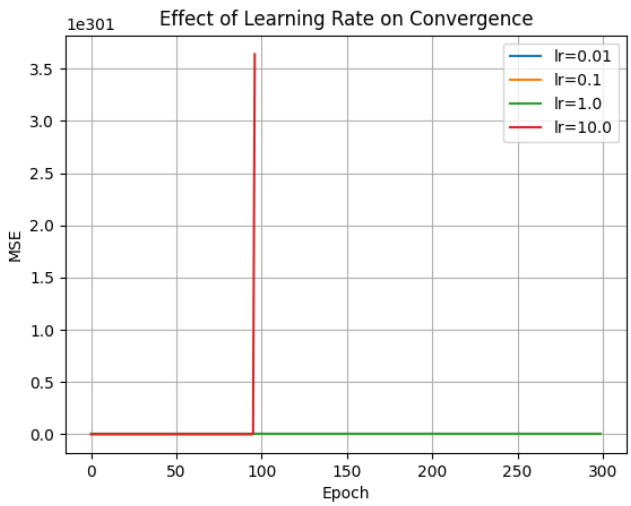

Figure 5: Convergence curves for different learning rates.

(d) Effect of Training Set Size

The amount of training data is varied from 10% to 100% in increments of 10%. The final validation MSE is plotted against training set size.

- Performance improves with more data.
- Gains diminish beyond a certain dataset size.

This behavior is typical for linear models with limited representational capacity.

Question 6: Multi layered Perceptron with Backpropagation

6.1 Theory

(a) Vanishing Gradient Problem with Sigmoid Activations

Consider a deep feedforward neural network with L layers. For layer ℓ, define the affine transformation and activation as

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \sigma(z^{(\ell)}),$$

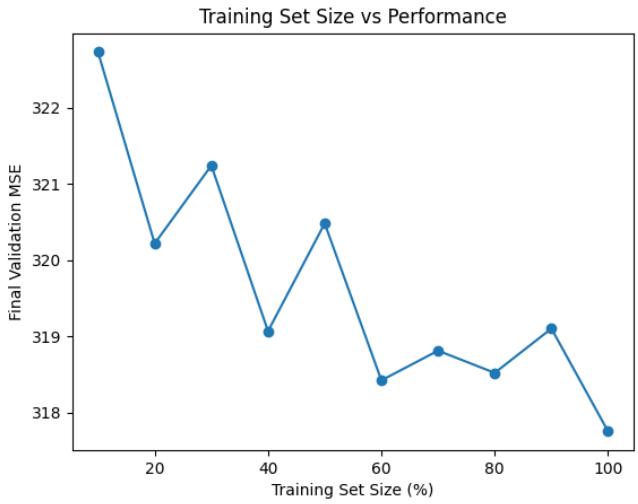

Figure 6: Final validation MSE versus training set size.

where $\sigma(\cdot)$ is the sigmoid activation function

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

Let J denote the loss function at the network output. During backpropagation, we are interested in computing gradients of the form

$$\delta^{(\ell)} := \frac{\partial J}{\partial z^{(\ell)}}.$$

Application of the Chain Rule For any hidden layer ℓ, the loss J depends on $z^{(\ell)}$ only through the activations of the next layer $a^{(\ell)}$. Applying the chain rule,

$$\frac{\partial J}{\partial z^{(\ell)}} = \frac{\partial J}{\partial a^{(\ell)}} \odot \frac{\partial a^{(\ell)}}{\partial z^{(\ell)}}.$$

Since $a^{(\ell)} = \sigma(z^{(\ell)})$, we have

$$\frac{\partial a^{(\ell)}}{\partial z^{(\ell)}} = \sigma'(z^{(\ell)}).$$

Next, J depends on $a^{(\ell)}$ through $z^{(\ell+1)} = W^{(\ell+1)} a^{(\ell)} + b^{(\ell+1)}$. Thus,

$$\frac{\partial J}{\partial a^{(\ell)}} = \left(W^{(\ell+1)}\right)^\top \frac{\partial J}{\partial z^{(\ell+1)}}.$$

Combining the above expressions yields

$$\delta^{(\ell)} = \left(W^{(\ell+1)}\right)^\top \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)}).$$

Derivative of the Sigmoid The derivative of the sigmoid function is

$$\sigma'(t) = \sigma(t)\big(1 - \sigma(t)\big).$$

Since $0 < \sigma(t) < 1$ for all $t \in \mathbb{R}$, it follows that

$$0 < \sigma'(t) \leq \frac{1}{4},$$

with the maximum value $\frac{1}{4}$ attained at $t = 0$.

Recursive Gradient Propagation Applying the backpropagation relation repeatedly from layer L to layer 1, we obtain

$$\delta^{(L-1)} = \left(W^{(L)}\right)^\top \delta^{(L)} \odot \sigma'(z^{(L-1)}),$$

$$\delta^{(L-2)} = \left(W^{(L-1)}\right)^\top \delta^{(L-1)} \odot \sigma'(z^{(L-2)}),$$

and so on.

Substituting recursively gives

$$\delta^{(1)} = \left(\prod_{\ell=2}^{L} \left(W^{(\ell)}\right)^\top \operatorname{diag}\big(\sigma'(z^{(\ell-1)})\big)\right) \delta^{(L)}.$$

Norm Bound on the Gradient Taking norms on both sides and upper-bounding the norm of a product by the product of norms, we obtain

$$\|\delta^{(1)}\| \leq \prod_{\ell=2}^{L} \left\|\left(W^{(\ell)}\right)^\top\right\| \cdot \|\sigma'(z^{(\ell-1)})\| \cdot \|\delta^{(L)}\|.$$

Since $\|\sigma'(z)\| \leq \frac{1}{4}$ for all layers, we obtain

$$\|\delta^{(1)}\| \leq \left(\frac{1}{4}\right)^{L-1} \left(\prod_{\ell=2}^{L} \|W^{(\ell)}\|\right) \|\delta^{(L)}\|.$$

Vanishing Gradient Effect As the network depth L increases,

$$\left(\frac{1}{4}\right)^{L-1} \longrightarrow 0.$$

Hence, gradients propagated to early layers decay exponentially with depth. This causes weight updates in early layers to become extremely small, making learning slow or ineffective in deep networks. This phenomenon is known as the *vanishing gradient problem.*

(b) Effect of Randomly Dropped Hidden Units on the Chain Rule

Suppose that at each hidden layer ℓ, each unit is independently dropped with probability 0.5. Let

$$m^{(\ell)} \in \{0,1\}^{d_\ell}$$

denote a random binary mask, where

$$\mathbb{P}\big(m_i^{(\ell)} = 1\big) = 0.5, \quad \mathbb{P}\big(m_i^{(\ell)} = 0\big) = 0.5.$$

Forward Pass with Dropped Units The affine transformation at layer ℓ is unchanged:

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}.$$

The activation is modified by elementwise masking:

$$a^{(\ell)} = m^{(\ell)} \odot \sigma(z^{(\ell)}).$$

Thus, for each unit i,

$$a_i^{(\ell)} = \begin{cases} \sigma(z_i^{(\ell)}), & m_i^{(\ell)} = 1, \\ 0, & m_i^{(\ell)} = 0. \end{cases}$$

Gradient with Respect to $z^{(\ell)}$ We define

$$\delta^{(\ell)} := \frac{\partial J}{\partial z^{(\ell)}}.$$

Applying the chain rule,

$$\frac{\partial J}{\partial z^{(\ell)}} = \frac{\partial J}{\partial a^{(\ell)}} \odot \frac{\partial a^{(\ell)}}{\partial z^{(\ell)}}.$$

Since

$$a^{(\ell)} = m^{(\ell)} \odot \sigma(z^{(\ell)}),$$

we obtain

$$\frac{\partial a^{(\ell)}}{\partial z^{(\ell)}} = m^{(\ell)} \odot \sigma'(z^{(\ell)}).$$

Therefore,

$$\delta^{(\ell)} = \frac{\partial J}{\partial a^{(\ell)}} \odot m^{(\ell)} \odot \sigma'(z^{(\ell)}).$$

Propagation of Gradients from the Next Layer The loss J depends on $a^{(\ell)}$ through the pre-activation

$$z^{(\ell+1)} = W^{(\ell+1)} a^{(\ell)} + b^{(\ell+1)}.$$

Applying the chain rule again,

$$\frac{\partial J}{\partial a^{(\ell)}} = \left(W^{(\ell+1)}\right)^\top \frac{\partial J}{\partial z^{(\ell+1)}} = \left(W^{(\ell+1)}\right)^\top \delta^{(\ell+1)}.$$

Substituting this into the previous expression yields

$$\delta^{(\ell)} = \left(W^{(\ell+1)}\right)^\top \delta^{(\ell+1)} \odot m^{(\ell)} \odot \sigma'(z^{(\ell)}).$$

Effect of the Mask on Gradient Flow For each unit i in layer ℓ,

$$\delta_i^{(\ell)} = m_i^{(\ell)} \, \sigma'(z_i^{(\ell)}) \left[\left(W^{(\ell+1)}\right)^\top \delta^{(\ell+1)}\right]_i.$$

Thus:

- If $m_i^{(\ell)} = 0$, then $\delta_i^{(\ell)} = 0$ and no gradient flows through unit i.
- If $m_i^{(\ell)} = 1$, gradient propagation is identical to the fully connected case.

Conclusion The chain rule itself is unchanged. However, it is applied only along the active paths defined by the random masks. At each iteration, backpropagation operates on a random subgraph of the original network, introducing stochasticity and acting as a form of implicit regularization.

6.3 : Architecture Ablation

6.3.2 Width vs. Depth Performance Relationship

The width and depth of a neural network affect performance in different ways. Width controls the *number of parallel features* learned at each layer, while depth controls the *hierarchical composition* of features.

From the depth ablation (Section 6.3.1), increasing depth initially improves performance by allowing the network to represent more complex functions. However, beyond a small number of layers, deeper networks show diminishing returns and are harder to optimize, leading to slower convergence and larger train–validation gaps.

From the width ablation, increasing width improves performance by increasing model capacity, but the number of parameters grows quadratically with width. As a result, test accuracy saturates after a certain width, while computational cost and overfitting risk continue to increase.

Comparing the two, depth is generally more parameter-efficient, as deeper networks can represent complex functions with fewer parameters than very wide shallow networks. However, depth introduces optimization difficulties such as vanishing gradients, while width primarily affects generalization and model size.

Overall, the experiments suggest that a *moderately deep and moderately wide* architecture provides the best trade-off between performance, generalization, and computational efficiency.

6.5 Comparison of Different Loss Functions

To study the effect of the choice of loss function on model performance, three commonly used regression losses were evaluated: Mean Squared Error (MSE), Mean Absolute Error (MAE), and Huber loss. The training and validation loss curves for each case are shown below.

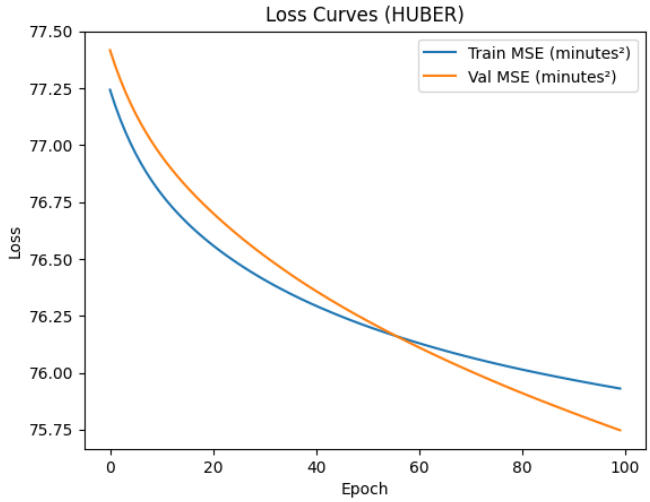

Figure 7: Training and validation loss curves using Huber loss

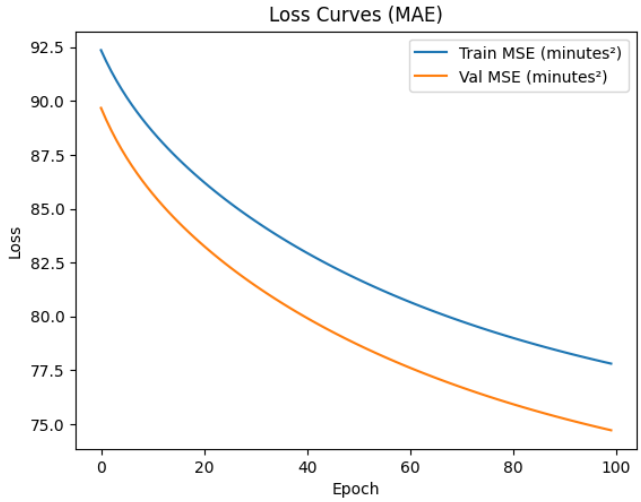

Figure 8: Training and validation loss curves using MAE loss

Discussion The MSE loss penalizes large errors quadratically, which makes it highly sensitive to outliers. This results in rapidly increasing loss values and can cause the training process to be dominated by a few large residuals. MAE, on the other hand, penalizes errors linearly and is

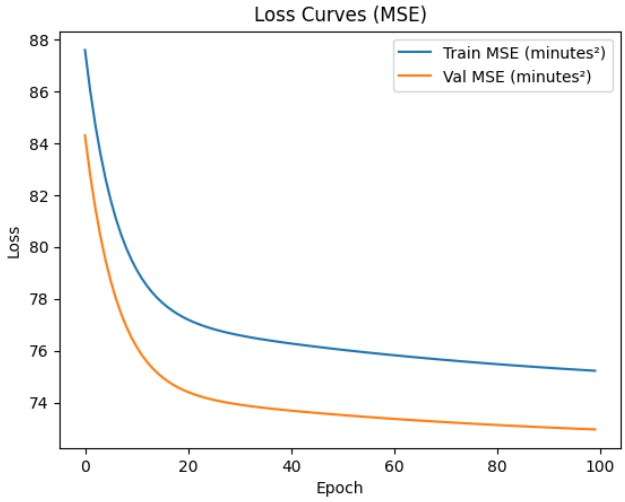

Figure 9: Training and validation loss curves using MSE loss

more robust to outliers, but it leads to less smooth optimization due to its non-differentiability at zero.

Huber loss combines the advantages of both MSE and MAE: it behaves like MSE for small errors (ensuring smooth optimization) and like MAE for large errors (providing robustness to outliers). From the loss curves, Huber loss shows stable training behavior and consistently lower validation loss compared to MSE, while being smoother than MAE.

Conclusion Among the three loss functions, **Huber loss is the most suitable choice** for this problem, as it provides a good balance between sensitivity to small errors and robustness to outliers, resulting in stable training and better generalization performance.

6.7 Success Analysis

(a) PCA of Hidden Layer Activations The activations from the penultimate layer of the best MLP (depth 2, width 64) were projected to 2D using PCA and coloured by binned mess duration. Clusters in the PCA plot indicate that the network has learned a structured internal representation of the input features, separating samples by output range even without explicit class supervision.

(b) MLP vs. ADALINE Comparison

Model	**Test Acc ($\pm$5 min)**	**Test MSE**	**Train Time (s)**
ADALINE	0.6707	27.54	0.024
MLP [input, 64, 64, output]	0.4267	82.34	1.704

Table 1: ADALINE vs. MLP on IIT-H Mess Duration prediction.

Surprisingly, ADALINE outperforms the MLP on this task. This is likely because the mess duration target has a strong linear component — ADALINE's least-squares solution directly

minimises MSE and generalises well, while the MLP may be overfitting the small dataset given its much larger parameter count. The MLP also takes $\sim 70\times$ longer to train. This highlights that model complexity does not always yield better performance, particularly on low-sample tabular regression tasks.

7 Neural Network Features for Kernel Methods (Advanced)

Neural networks can be interpreted as *learned feature maps* that enable linear methods to solve nonlinear problems, analogous to kernel methods.

7.1

(a) Kernel Methods Definition (Valid Kernel). A function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a valid kernel if there exists a feature map

$$\phi : \mathcal{X} \to \mathcal{H}$$

into a Hilbert space $\mathcal{H}$ such that

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}.$$

Equivalently, for any $\{x_i\}_{i=1}^n \subset \mathcal{X}$, the Gram matrix

$$K_{ij} = k(x_i, x_j)$$

must satisfy

$$K \succeq 0 \quad \text{(positive semidefinite)}.$$

Mercer's Theorem. If $k(x, x')$ is continuous, symmetric, and positive semidefinite on a compact domain, then it admits the expansion

$$k(x, x') = \sum_{m=1}^{\infty} \lambda_m \psi_m(x) \psi_m(x'),$$

where

$$\lambda_m \geq 0,$$

and $\{\psi_m\}$ form an orthonormal basis in L^2.

Thus,

$$\phi(x) = \left(\sqrt{\lambda_1} \psi_1(x), \sqrt{\lambda_2} \psi_2(x), \dots \right),$$

and

$$k(x, x') = \langle \phi(x), \phi(x') \rangle.$$

Kernel Trick. For linear models

$$f(x) = w^\top \phi(x),$$

using the representer theorem,

$$w = \sum_{i=1}^{n} \alpha_i \phi(x_i),$$

so

$$f(x) = \sum_{i=1}^{n} \alpha_i k(x_i, x),$$

without explicitly computing $\phi(x)$.

(b) Neural Networks as Feature Maps Consider a neural network with one hidden layer:

$$h(x) = \sigma(Wx + b),$$

$$f(x) = v^\top h(x).$$

Define the learned feature map

$$\phi_\theta(x) := h(x).$$

Then

$$f(x) = v^\top \phi_\theta(x),$$

which is linear in the learned feature space.

Connection to Kernels.

Define the implicit kernel:

$$k_\theta(x, x') = \langle \phi_\theta(x), \phi_\theta(x') \rangle = h(x)^\top h(x').$$

Thus:

$$\text{Kernel methods: } \phi \text{ fixed, } w \text{ learned}$$

$$\text{Neural networks: } \phi_\theta \text{ learned, } w \text{ learned}$$

Key Difference:

$$\text{Kernel trick } \Rightarrow \text{ implicit infinite-dimensional } \phi$$

$$\text{Neural nets } \Rightarrow \text{ explicit finite-dimensional learned } \phi_\theta$$

Hence, a hidden layer acts as a *data-adaptive feature map*, making neural networks learn a task-specific kernel:

$$k_\theta(x, x') = \phi_\theta(x)^\top \phi_\theta(x').$$